

```

1 import numpy as np
2 # --- PART A: Perceptron ---
3 class Perceptron:
4     def __init__(self, input_size, lr=0.1, epochs=10):
5         self.weights = np.zeros(input_size + 1) # Includes bias
6         self.lr = lr
7         self.epochs = epochs
8     def activate(self, x):
9         return 1 if x >= 0 else 0
10    def train(self, X, y):
11        for _ in range(self.epochs):
12            for xi, target in zip(X, y):
13                inputs = np.insert(xi, 0, 1) # Insert bias unit
14                prediction = self.activate(np.dot(inputs, self.weights))
15                self.weights += self.lr * (target - prediction) * inputs
16    def predict(self, X):
17        inputs = np.insert(X, 0, 1)
18        return self.activate(np.dot(inputs, self.weights))
19 # --- PART B: Backpropagation ---
20 def sigmoid(x): return 1 / (1 + np.exp(-x))
21 def sigmoid_derivative(x): return x * (1 - x)
22 def train_backpropagation(X, y, epochs=5000, lr=0.1):
23     input_layer_size = X.shape[1]
24     hidden_layer_size = 3

```

```

22 def train_backpropagation(x, y, epochs=1000, lr=0.1):
23     input_layer_size = X.shape[1]
24     hidden_layer_size = 3
25     output_layer_size = 1
26
27     wh = np.random.uniform(size=(input_layer_size, hidden_layer_size))
28     wout = np.random.uniform(size=(hidden_layer_size, output_layer_size))
29
30     for _ in range(epochs):
31         # Forward pass
32         hidden_layer_input = np.dot(X, wh)
33         hidden_layer_activations = sigmoid(hidden_layer_input)
34         output_layer_input = np.dot(hidden_layer_activations, wout)
35         predicted_output = sigmoid(output_layer_input)
36
37         # Backpropagation
38         error = y - predicted_output
39         d_predicted_output = error * sigmoid_derivative(predicted_output)
40
41         hidden_layer_error = d_predicted_output.dot(wout.T)
42         d_hidden_layer = hidden_layer_error *
43 sigmoid_derivative(hidden_layer_activations)
44
45         wout += hidden_layer_activations.T.dot(d_predicted_output) * lr
46         wh += X.T.dot(d_hidden_layer) * lr
47

```

```

45     wout += hidden_layer_activations.T.dot(d_predicted_output) * lr
46     wh += X.T.dot(d_hidden_layer) * lr
47
48     return predicted_output
49 # Example Usage
50 if __name__ == "__main__":
51     print("--- Training Perceptron (AND Gate) ---")
52     X_and = np.array([[0,0], [0,1], [1,0], [1,1]])
53     y_and = np.array([0, 0, 0, 1])
54     p = Perceptron(input_size=2)
55     p.train(X_and, y_and)
56     print("Prediction for [1, 1]:", p.predict(np.array([1, 1])))
57     print("\n--- Training Backpropagation (XOR Gate) ---")
58     X_xor = np.array([[0,0], [0,1], [1,0], [1,1]])
59     y_xor = np.array([[0], [1], [1], [0]])
60     outputs = train_backpropagation(X_xor, y_xor)
61     print("Final outputs after 5000 epochs:\n", outputs)
62

```

Program 8: Perceptron and Backpropagation Neural Network

```
--- Training Perceptron (AND Gate) ---
```

```
Prediction for [1, 1]: 1
```

```
--- Training Backpropagation (XOR Gate) ---
```

```
Final outputs after 5000 epochs:
```

```
[[0.44693893]  
 [0.50183621]  
 [0.57727151]  
 [0.53496814]]
```

```
Zainab khatoon 24becs173
```
